

Architecture for a low-traffic, high-trust B2B site

The macro shape is forced by the project's constraints. These are the internal decisions that remain — and the one that's genuinely expensive to defer.

00 What this architecture is called

Architecture isn't one label — it's a stack of decisions, each with a proper name. Naming them precisely matters more than picking a fashionable umbrella term.

DIMENSION	PATTERN	MEANING HERE
CMS delivery model	Coupled (traditional) CMS	The CMS handles both content management and presentation. Not headless (API-only) and not decoupled (hybrid)
Deployment topology	Monolith	One deployable unit, one host, one backup story
Application pattern	MVC	ASP.NET Core MVC as Umbraco implements it: <code>RenderController</code> / <code>SurfaceController</code> , Razor views, <code>ModelsBuilder</code> models
Rendering strategy	SSR	HTML assembled server-side; no client-side hydration layer
Code organization	Package-by-layer	Folders by technical concern — Views, Services, Controllers, Middleware
Caching	Reverse-proxy / edge caching	CDN acts as a caching reverse proxy in front of the origin (\$02)

One-phrase version: *a coupled CMS architecture — server-rendered MVC monolith with edge caching.*

TERMINOLOGY CORRECTION

An earlier draft of this document called it a "modular monolith." That was imprecise. A modular monolith implies enforced internal module boundaries — vertical slices or bounded contexts within one deployable. This solution is organized by technical concern, which is **package-by-layer**. Different pattern; the accurate term is used throughout from here.

Why not DDD, Clean, Onion, or Hexagonal

These are the patterns a senior .NET developer would reach for by default, so the decision to skip them should be explicit rather than accidental.

Not Domain-Driven Design

REJECTED – NO DOMAIN TO DRIVE

DDD manages complexity in a business domain: aggregates, invariants, ubiquitous language, bounded contexts. This site has no business domain. Its entities are pages and documents; its behaviour is rendering them. There are no business rules to encode and no invariants to protect, so DDD's structure is cost without corresponding complexity.

Not Clean / Onion / Hexagonal Architecture

REJECTED – THE FRAMEWORK IS THE APPLICATION

These patterns exist to keep a domain core independent of frameworks and infrastructure, so the database or web framework can be swapped without touching business logic. Here Umbraco supplies the content model, routing, rendering and editor UI — it *is* the application. Abstracting away from it would mean building an indirection layer over the very thing providing the value, to enable a swap that will never occur. The pattern's benefit doesn't apply.

One construct resembles ports-and-adapters — `ISender` as an interface with a SendGrid implementation — but that is ordinary dependency injection for testability, not hexagonal architecture, and shouldn't be described as such.

WHEN THIS WOULD CHANGE

If the deferred buyer RFQ portal is ever built (quotes, statuses, buyer accounts, approval flow), that *is* real domain logic with real invariants. At that point the correct move is a properly layered module with its own bounded context **inside** this monolith — not a rewrite, and not retrofitting layering across the

brochure pages that will never need it. The trigger is the arrival of genuine domain complexity, not a growth in page count.

01 Why the alternatives lose

Stated once so it isn't re-litigated later. Each is rejected on a specific requirement, not on preference.

ALTERNATIVE	WHY IT'S REJECTED HERE
Headless (Umbraco + Astro/Next.js)	Two hosts, two deploy pipelines, fast-churning JS ecosystem. Doubles cost and violates "maintainable alone, without AI." Earns its place at omnichannel scale this site doesn't have.
Static SSG + git-based CMS	Cheapest and most secure, but git/markdown concepts leak into the owner's editing experience, and it isn't .NET. Fails the two hardest requirements simultaneously.
Hand-rolled ASP.NET Core + custom admin	Months spent rebuilding a worse Umbraco backoffice. The CMS is the part worth not writing.
Layered Clean Architecture / DDD inside the app	Correct for a transactional system; overhead for a content site with no complex domain logic. See §00 for the full reasoning and the trigger that would change it.

Conclusion: a coupled CMS architecture — server-rendered MVC monolith on Umbraco 18 / .NET 10, single deployable unit, edge-cached.

02 Edge-first caching

The decision that makes budget hosting viable without the visitor ever experiencing budget hosting.

Content changes roughly monthly. Traffic is low, but the individual visitor matters enormously — a buyer's compliance officer or bank clicking through mid-due-

diligence. On a shared host, cold starts and CPU limits are real. Caching rendered HTML at the edge decouples the visitor's experience from the origin's performance entirely.

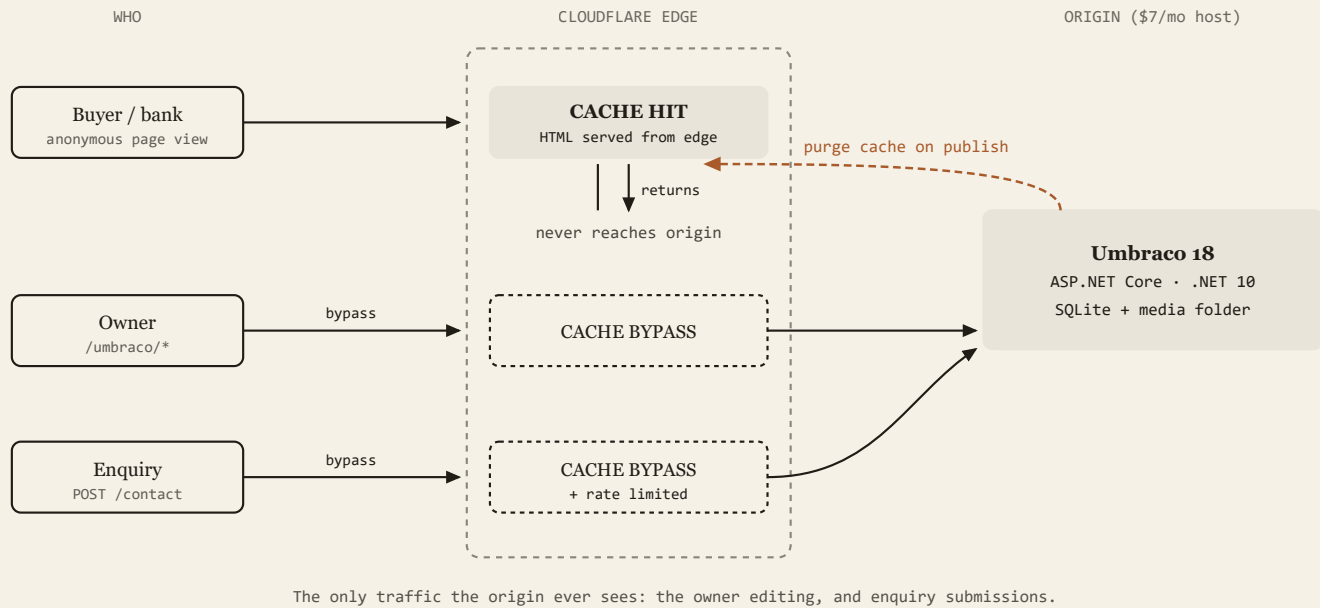


Fig. 1 — Three request paths. Public page views terminate at the edge, so origin performance never reaches the visitor; the backoffice and the single write endpoint deliberately bypass cache. Publishing purges the edge so content changes appear immediately.

WHAT THIS BUYS

Near-static performance and meaningful DDoS resilience on the cheapest hosting tier, plus headroom if the site is ever linked from a trade publication. It also means the choice between Tier A and Tier B hosting stops being a performance decision and becomes purely an operational-convenience one.

03 The decision not to defer

Everything else here can be changed later at reasonable cost. This one can't.

Enable culture variance from day one — even though bilingual is a v2 item

DECIDE NOW · EXPENSIVE TO RETROFIT

Roughly 100% of the company's recorded buyers are China-based, so an English/Simplified-Chinese site is a plausible near-term requirement rather than a hypothetical. In Umbraco, whether content varies by culture is a per-document-type and per-property setting, and enabling it retrospectively carries documented migration edge cases — notably that culture-varying properties are skipped when the parent content type is invariant, which lands precisely on the composition-based model planned here (`seoComposition` applied to otherwise-invariant types).

Recommendation: turn on *Allow vary by culture* for the document types and their editable properties at build time, with English as the only active language. The near-term cost is close to zero — one language node and slightly different value access in Razor. The payoff is that adding Chinese later becomes "add a language, translate the content" instead of a content-model migration against live production content.

04 Remaining decisions

Server-rendered Razor, no SPA layer

DECIDED

SEO works without effort, there's no hydration cost on mobile — links get opened from WhatsApp on Philippine and Chinese mobile networks — and it's one fewer ecosystem to maintain. ModelsBuilder supplies strongly-typed content models so views stay readable.

Constrained content model, not a free-form page builder

DECIDED

Rigidity is a feature on a compliance site. Fixed document types with an approved set of Block List blocks per page type: the owner edits text, swaps images, publishes news, reorders approved blocks — and cannot restructure the Certifications page into something that looks broken while a bank is reviewing it.

Content model versioned in git via uSync

DECIDED

uSync 18.0.3 supports Umbraco 18. Document types, data types and templates serialize to disk and get committed, so the schema is versioned alongside the Razor views that depend on it and dev→prod deployment is a file sync rather than manual backoffice clicking. Content itself is *not* synced — production content belongs to the owner.

Single write path, treated as an invariant

DECIDED · GUARD IT

Publicly the site is read-only except one endpoint (the contact form POST). That's an unusually small attack surface and worth protecting as an architectural property, not an accident. Any future feature that adds a write path — RFQ portal, buyer login — is a genuine architectural change requiring a security rethink, not just another page. The backoffice sits behind Cloudflare, ideally IP-restricted.

Backup/DR falls out of the SQLite choice

CONSEQUENCE, NOT A SEPARATE DECISION

The site's entire state is one database file plus the media folder. Backup is copying two things; restore is putting them back. That operational simplicity is a real architectural benefit of SQLite here, not merely a cost saving.

05 Solution structure

One project, organized by concern — not layered into artificial assemblies.

FOLDER	CONTAINS
Views/	Razor templates, one per document type, plus shared layout and partials (<code>_SeoHead.cshtml</code>)
Services/	<code>ISender</code> , sitemap generation — plain C#, unit-testable, no Umbraco coupling where avoidable

Controllers/	<code>ContactSurfaceController</code> — the one write path
Composers/	Umbraco DI registration (<code>IComposer</code> implementations)
Middleware/	Security-header middleware
uSync/	Serialized content model — committed to git